

PRD (Product Requirement Document)

ELKash ERP System

1. Product Overview

Product Name

ELKash ERP

Product Type

Web-Based Modular ERP + POS System

Product Vision

ELKash bertujuan menjadi ERP modular modern untuk bisnis cafe, coffee shop, dan small restaurant dengan fokus pada:

- integrasi proses bisnis,
- stabilitas transaksi,
- kemudahan operasional,
- dan scalable architecture.

Sistem dirancang sebagai realistic enterprise-grade portfolio project dengan modular architecture dan clean business workflow.

2. Product Objectives

Primary Objectives

Operational Objectives

- Mempercepat proses transaksi POS
- Mengurangi mismatch inventory
- Mengurangi proses manual reporting
- Menyediakan centralized business data

Technical Objectives

- Membangun modular ERP architecture
- Menyediakan scalable REST API backend
- Menerapkan secure authentication menggunakan Laravel Sanctum
- Menyediakan maintainable frontend architecture berbasis Nuxt 3

Product Objectives

- Menyediakan responsive POS experience
 - Menyediakan dashboard analytics real-time
 - Menyediakan inventory monitoring yang akurat
 - Menyediakan reporting system yang cepat dan exportable
-

3. Product Scope

V1 Included Scope

Auth Module

Features:

- Login
- Logout
- Session management
- Role Based Access Control (RBAC)
- Protected route middleware
- Permission validation

POS Module

Features:

- Product grid
- Category filter
- Search product
- Cart management
- Checkout flow
- Payment processing
- Invoice generation
- Transaction history

Inventory Module

Features:

- Product management
- Category management
- Stock adjustment
- Stock movement logs
- Low stock monitoring
- Inventory synchronization

Dashboard Module

Features:

- Revenue summary
- Sales statistics
- Best seller products
- Low stock alerts
- Daily transaction summary
- Monthly analytics

Reporting Module

Features:

- Daily sales report
- Monthly sales report
- Inventory report
- Cashier report
- Export PDF
- Export Excel

4. Out of Scope (V1)

The following features are excluded from initial release:

- Accounting module
- Payroll system
- HRIS
- CRM
- Mobile application
- Offline-first support
- Multi-tenant architecture
- WebSocket real-time synchronization

- AI recommendation system
 - Marketplace integration
 - Loyalty point system
 - Kitchen Display System (KDS)
-

5. Target Users

Super Admin

Responsibilities:

- Manage roles & permissions
- System configuration
- Monitor operational activity

Permissions:

- Full system access
-

Admin

Responsibilities:

- Manage inventory
- Manage products
- Generate reports

Permissions:

- Inventory access
 - Reporting access
 - Product management
-

Cashier

Responsibilities:

- Handle customer transactions
- Process payments
- Generate receipts

Permissions:

- POS access only
-

Manager

Responsibilities:

- Monitor business performance
- Analyze reports
- Monitor inventory

Permissions:

- Dashboard access
 - Reporting access
 - Read-only operational access
-

6. User Flow

Login Flow

User opens login page → Input email/password → API authentication → Sanctum session created → User permissions loaded → Redirect to dashboard

POS Transaction Flow

Cashier selects product → Add item to cart → Validate stock → Checkout → Payment confirmation → Save transaction → Reduce inventory stock → Generate invoice → Update dashboard analytics

Inventory Flow

Admin updates product stock → Validate adjustment → Record stock movement → Update inventory state → Trigger low stock detection

Reporting Flow

Manager selects report type → System aggregates transactional data → Generate report → Export PDF/Excel

7. Functional Requirements

7.1 Authentication Module

Functional Requirements

ID	Requirement
AUTH-01	User must login before accessing system
AUTH-02	Session managed using Laravel Sanctum
AUTH-03	Unauthorized users redirected to login
AUTH-04	Route access validated by RBAC
AUTH-05	Logout invalidates active session

UI Requirements

- Clean enterprise login page
 - Dark/light mode support
 - Responsive mobile layout
 - Password visibility toggle
-

7.2 POS Module

Functional Requirements

ID	Requirement
POS-01	Cashier can browse products
POS-02	Product search must support keyword filtering
POS-03	Cart updates in real-time
POS-04	Checkout blocked when stock unavailable
POS-05	Successful transaction reduces inventory

ID	Requirement
POS-06	Invoice generated after payment
POS-07	Transaction history searchable

UI Requirements

- Fast product grid
- Sticky cart panel
- Responsive tablet layout
- Optimized cashier workflow
- Large touch-friendly buttons

Business Rules

- Empty cart cannot checkout
- Stock validation mandatory
- Completed transactions immutable

7.3 Inventory Module

Functional Requirements

ID	Requirement
INV-01	Admin can create product
INV-02	Admin can update stock
INV-03	System records stock movement
INV-04	Low stock alerts generated automatically
INV-05	Product deletion uses soft delete

UI Requirements

- Table-based inventory management
- Quick stock adjustment modal
- Low stock highlighting
- Filter & sorting support

Business Rules

- Historical transactions preserved
- Stock changes require audit logs

7.4 Dashboard Module

Functional Requirements

ID	Requirement
DASH-01	Dashboard displays revenue summary
DASH-02	Dashboard displays sales chart
DASH-03	Dashboard displays low stock alerts
DASH-04	Dashboard displays best seller products

UI Requirements

- Modern enterprise dashboard
 - Card-based analytics
 - Responsive charts
 - Fast initial loading
-

7.5 Reporting Module

Functional Requirements

ID	Requirement
REP-01	Generate daily sales report
REP-02	Generate monthly sales report
REP-03	Generate inventory report
REP-04	Export PDF
REP-05	Export Excel

UI Requirements

- Report filter section
 - Export buttons
 - Date range picker
 - Data table pagination
-

8. Non-Functional Requirements

Performance

- Checkout completion < 10 seconds
- Dashboard load time < 3 seconds
- Inventory update latency < 3 seconds

Security

- Sanctum-based authentication
- Protected API endpoints
- Role-based access validation
- Frontend never directly accesses database

Scalability

- Modular architecture
- API-first design
- Expandable module structure
- Reusable composables & services

Reliability

- Transaction success rate > 99%
- Stock synchronization accuracy > 95%

Maintainability

- Type-safe frontend architecture
 - Clean folder structure
 - Reusable UI components
 - Standardized API response structure
-

9. Technical Architecture

Frontend Stack

- Nuxt 3
- Vue 3
- TypeScript
- Pinia
- TailwindCSS

- Shadcn Vue

Backend Stack

- Laravel REST API
- Laravel Sanctum
- PostgreSQL (Supabase)
- Redis (Optional)

Deployment

Frontend

- Netlify
- GitHub auto deploy

Backend

- Ubuntu VPS
- Nginx
- SSL Let's Encrypt
- Supervisor/PM2

10. Frontend Architecture

Proposed Frontend Structure

```

`bash id="g4u6p0" src/ |—— modules/ | |—— auth/ | |—— pos/ | |—— inventory/ | |——
dashboard/ | |—— reporting/ | |—— shared/ |—— components/ |—— composables/ |—— stores/
|—— services/ |—— middleware/ |—— types/ |—— utils/

```

```

## State Management
Pinia digunakan untuk:

```

- auth state
- cart state
- inventory state
- dashboard state
- reporting filters

```

## API Layer
Menggunakan centralized API service:

```

- axios/fetch wrapper

```
- token handling
- error normalization
- response typing

---

# 11. Backend Architecture

## Laravel Responsibilities
- Authentication authority
- Business logic
- Validation
- RBAC
- Transaction processing
- Reporting aggregation
- Inventory authority

## API Design

REST API versioning:

```bash id="9q5o1u"
/api/v1/
```

## Backend Principles

- Thin controller
- Service layer architecture
- Request validation
- Repository pattern (optional)
- Transaction-safe inventory update

---

## 12. Database Requirements

### Core Tables

#### Auth

- users
- roles
- permissions
- role\_permissions

## Inventory

- products
- categories
- stock\_movements

## POS

- transactions
- transaction\_items
- payments

## Reporting

- aggregated\_reporting\_tables (optional future optimization)
- 

# 13. Audit Requirements

System should log:

- login activity
- failed authentication
- stock adjustments
- transaction cancellation
- product updates
- permission changes

Audit logs should:

- contain actor & timestamp
  - immutable from frontend
  - support operational tracing
- 

# 14. Success Metrics

## Business Metrics

Metric	Target
Checkout completion time	< 10 sec
Stock synchronization accuracy	> 95%
Daily report generation	< 5 sec

Metric	Target
Manual reporting reduction	70%
POS transaction success rate	> 99%

## Technical Metrics

Metric	Target
API response consistency	Stable
Frontend responsiveness	High
Architecture modularity	Maintainable
Authentication reliability	> 99%

---

# 15. Release Plan

## Phase 1

- Authentication
- RBAC
- POS Core
- Inventory Core

## Phase 2

- Dashboard
- Reporting
- Export Features

## Phase 3

- Optimization
  - Audit Logging
  - Security Hardening
  - Performance Improvement
-

## 16. API Contract Philosophy

### API Response Standard

All API responses must follow standardized response structure to ensure consistency between frontend and backend communication.

#### Success Response

```
``json id="4j0d0u" { "success": true, "message": "Transaction created", "data": {} }
```

### Error Response

```
``json id="w9j2eq"
{
 "success": false,
 "message": "Stock unavailable",
 "errors": {}
}
```

### API Principles

- API responses must be predictable and typed
  - Business validation errors must use standardized error structure
  - HTTP status codes should reflect request outcome correctly
  - Validation errors should be field-specific when possible
  - API versioning required for future scalability
- 

## 17. State Persistence Rules

### Persistent State

The following states should persist across browser refresh/session:

- auth session
- user preferences
- theme preferences
- sidebar state

## Temporary State

The following states should remain temporary in memory:

- cart state
- report filters
- dashboard widget filters
- temporary modal state

## State Management Principles

- Global state managed using Pinia
  - Avoid unnecessary persistent storage
  - Sensitive authentication state must not expose secure backend data
  - Persistent state should be minimized for security and maintainability
- 

# 18. Transaction Integrity Rules

To maintain operational consistency and ERP reliability:

- Inventory deduction must occur inside database transaction
- Failed payment must not modify inventory
- Duplicate checkout submission must be prevented
- Transaction rollback required on critical failure
- Partial transaction persistence is prohibited
- Transaction records become immutable after successful checkout

## Backend Enforcement

Laravel backend acts as the single source of truth for:

- stock validation
  - payment validation
  - transaction persistence
  - rollback handling
- 

# 19. UI Consistency Guidelines

## Design Principles

- Use consistent spacing scale
- Use reusable table components

- Use standardized modal patterns
- Use centralized form validation UI
- Use consistent status badge colors
- Use reusable card components
- Use centralized typography hierarchy

## UI Component Strategy

Shared UI components should be reusable across modules:

- tables
- forms
- dialogs
- badges
- alerts
- dropdowns
- pagination

## UX Direction

The UI should reflect:

- enterprise minimalism
  - operational clarity
  - fast interaction flow
  - responsive layouts
  - low cognitive load
- 

# 20. Error Handling UX

## Error Handling Principles

- Validation errors must be human-readable
- Failed checkout should preserve cart state
- System errors should provide retry action
- Loading states required for all critical actions
- Error alerts should remain non-blocking where possible



## UX Error Categories

### Validation Errors

Examples:

- invalid form input
- stock unavailable
- insufficient permissions

Behavior:

- inline validation message
- highlighted invalid fields

### System Errors

Examples:

- server unavailable
- timeout
- failed export

Behavior:

- retry button
  - persistent notification
  - graceful fallback UI
- 

## 21. Device Support

### Primary Devices

System optimized primarily for:

- Desktop
- Tablet landscape

### Secondary Devices

Limited support:

- Mobile admin monitoring
- Mobile dashboard access

## Device Strategy

POS workflows prioritize:

- wide layouts
- fast cashier interaction
- tablet-friendly touch targets

Complex operational workflows are not prioritized for small-screen mobile devices in V1.

---

## 22. Reporting Principles

### Reporting Philosophy

- Reports generated from transactional records
- Aggregation optimized for performance
- Reports must remain consistent with operational data
- Reporting modules consume validated business data only

### Export Principles

Supported exports:

- PDF
- Excel

Future optimization:

- asynchronous export generation
- queued reporting jobs
- cached aggregated datasets

### Reporting Integrity

- Reports should remain reproducible from historical data
  - Reporting data should not directly modify transactional records
-

## 23. Queue Candidate Operations

### Potential Async Operations

The following operations are candidates for queue/background processing in future versions:

- PDF export
- audit logging
- report generation
- notification dispatch
- email sending
- large dataset aggregation

### Queue Philosophy

Queue implementation should:

- improve responsiveness
- reduce blocking operations
- avoid premature complexity in V1

Redis queue implementation remains optional until operational scale requires asynchronous processing.

---

## 24. Security Principles

### Backend Security Philosophy

- Backend validates all business-critical operations
- Frontend permissions are UI-only convenience
- Sensitive actions require server authorization
- API rate limiting recommended
- Database access restricted to backend server only

### Authentication Principles

- Authentication centralized via Laravel Sanctum
- Session validation required for protected endpoints
- Unauthorized API requests must be rejected consistently

## Operational Security

Sensitive operations include:

- stock adjustment
- role modification
- transaction cancellation
- report exports
- permission management

These actions require validated authorization rules.

---

## 25. Data Lifecycle Rules

### Data Retention Principles

- Transactions are immutable after completion
- Products use soft delete
- Audit logs retained permanently
- Reports generated from historical records

### Historical Integrity

Historical operational data must remain:

- traceable
- reproducible
- auditable

### Soft Delete Philosophy

Soft delete preferred for:

- products
- categories
- users (optional future)
- inventory records

Hard delete prohibited for transactional business records.

---

## 26. Acceptance Criteria

### POS-04 Acceptance Criteria

#### Scenario

Stock Validation During Checkout

#### Given

Product stock = 0

#### When

Cashier attempts checkout

#### Then

- System blocks transaction
  - System displays stock unavailable message
  - Cart state remains preserved
  - Inventory data remains unchanged
- 

### AUTH-01 Acceptance Criteria

#### Scenario

Unauthorized Access Prevention

#### Given

User not authenticated

#### When

User accesses protected route

#### Then

- System redirects user to login page
  - Protected content not displayed
  - Session validation triggered
-

## INV-03 Acceptance Criteria

### Scenario

Stock Adjustment Logging

### Given

Admin updates stock quantity

### When

Stock successfully updated

### Then

- System records stock movement log
  - Updated inventory displayed
  - Audit data stores actor and timestamp
- 

## REP-04 Acceptance Criteria

### Scenario

PDF Export Generation

### Given

Manager generates sales report

### When

Export PDF selected

### Then

- System generates downloadable PDF
  - Report data matches selected filters
  - Export process handled without data corruption
-

## 27. Engineering Philosophy

ELKash prioritizes:

1. business process consistency
2. transaction reliability
3. maintainable architecture
4. operational clarity
5. scalable modularity

The system intentionally avoids:

- premature microservices
- unnecessary AI features
- overengineered infrastructure
- feature quantity over workflow quality

ERP systems fail more often from inconsistent workflows and unreliable data than from lack of features.

Therefore: data integrity and operational stability are prioritized above visual complexity or excessive optimization.